Build a semester project security audit checklist covering OWASP ASVS, test cases, and a compliance matrix

Computer
AppSec debugging runbook: 10 common vulnerabilities in student apps (SQLi, XSS, RLS bypass) and how to fix them
Computer
Show me how to map the remaining ASVS 5.0 requirements
Draft an automated CI security testing pipeline script
Walk through how to execute test case 03 on RLS bypass

Created a complete **ASVS 5.0-oriented Semester security toolkit**, including the audit checklist, vulnerability debugging runbook, requirement-mapping guide, CI workflow draft, and a detailed RLS cross-tenant bypass test walkthrough. The CI workflow is intentionally fail-closed until you pin action hashes, the scanner image digest, Supabase CLI version, and confirm the actual repository test-script names.

Files created

File	Purpose
`ASVS-5-SECURITY-AUDIT-CHECKLIST.md`	Security audit checklist spanning all 17 ASVS 5.0 chapters, release gates, test expectations, and compliance applicability matrix
`APPSEC-10-COMMON-VULNERABILITIES-RUNBOOK.md`	Debugging and remediation runbook for 10 common student-app vulnerabilities
`ASVS-5-REQUIREMENT-MAPPING-GUIDE.md`	Process to replace the earlier chapter-level matrix with requirement-level ASVS 5.0.0 evidence
`RLS-AC-03-CROSS-TENANT-TEST-WALKTHROUGH.md`	Step-by-step, safe test for cross-tenant RLS bypasses
`security.yml`	Draft GitHub Actions security pipeline
`semester-asvs-5-security-toolkit.zip`	Bundle containing all five files

The 10 vulnerability runbook

The runbook covers these ten student-app risks:

1. SQL injection through filters, search, imports, or raw query building.
2. Stored, reflected, and DOM-based XSS in profiles, messages, rich text, and admin views.
3. RLS bypass / BOLA, where one student or institution accesses another's data.
4. Broken role escalation through client-controlled role, owner, grant, or approval fields.
5. JWT/session flaws involving expired, forged, revoked, or stolen sessions.
6. IDOR in files, exports, attachment paths, signed URLs, and derived views.
7. Unsafe file upload, including dangerous types, spoofed MIME types, and public-object exposure.
8. CSRF and cross-origin state changes where cookie-based authentication is used.
9. SSRF through document import, link preview, URL fetches, or integration callbacks.
10. Secret leakage and insecure CI behavior, including privileged service keys exposed to browsers or unsafe GitHub Actions workflows.

For every vulnerability, the document supplies an authorized validation approach, primary remediation, and expected regression proof. Cross-tenant authorization matters most for Semester because a student-data platform must prevent both **cross-user** and **cross-institution** access through every path—not merely through the visible UI.

Mapping ASVS 5.0

ASVS 5.0 has 17 chapters and 345 requirements. The generated guide requires importing the official version-pinned OWASP release and recording, per requirement:

```
asvs_version
asvs_id
chapter
requirement_text
level
applicable
applicability_rationale
asset_or_feature
control_id
implementation_ref
test_id
test_type
evidence_uri
status
owner
reviewer
last_verified
next_review
exception_id
notes
```

The high-priority mapping order is:

1. **V8 Authorization** — RLS, role/capability checks, RPCs, storage, realtime, exports.
2. **V6/V7/V9/V10** — authentication, session revocation, JWT handling, OAuth/OIDC if used.
3. **V14 Data Protection** — student data inventory, retention, exports, backups, and logging.
4. **V4 API and Web Service** — route inventory, schemas, authorization, webhooks, and limits.
5. **V1/V2/V3** — encoding, validation/business rules, React/browser defenses.
6. **V5** — private file storage and signed URL handling.
7. **V13/V15/V16** — configuration, CI/CD, dependencies, defensive coding, logging/errors.
8. **V11/V12** — cryptography, key lifecycle, TLS, and integration trust.
9. **V17** — mark Not Applicable only after confirming WebRTC is absent.

ASVS 5.0 separates authorization into V8, API security into V4, data protection into V14, and logging/error handling into V16. It also adds separate areas for self-contained tokens, OAuth/OIDC, and WebRTC. [1] [2]

CI pipeline safeguards

The `security.yml` pipeline includes:

- Full-repository and Git-history secret scanning.
- Locked dependency install with lifecycle scripts disabled initially.
- Dependency vulnerability auditing.
- Type-checking, test execution, and a required repository SAST script.
- Local isolated Supabase startup, migration reset, and database policy test execution.
- A release evidence gate after `main` branch checks succeed.

It uses the ordinary `pull_request` trigger, read-only `GITHUB_TOKEN` access, and avoids deployment or service-role secrets. It explicitly avoids `pull_request_target` because GitHub warns that checking out or executing untrusted pull-request code under that privileged trigger can compromise the repository and its secrets. [3] [4]

Before activation, replace these placeholders:

```
REPLACE_WITH_REVIEWED_SHA
REPLACE_WITH_REVIEWED_DIGEST
REPLACE_WITH_REVIEWED_CLI_VERSION
```

Also verify that these scripts exist and should fail CI if missing:

```
npm run typecheck
npm run test:ci
npm run security:sast
supabase test db
```

RLS test case 03

AC-03 proves that a T1 user cannot access T2 data. It must be tested with distinct synthetic users and institutions, then repeated across direct API access, RPC functions, database policy enforcement, exports, file storage, derived/cached data, and realtime subscriptions.

The key setup is to use an application-facing database role and a simulated authenticated JWT context, not a database owner or service-role connection. Supabase's testing guidance specifically calls for setting the local role and JWT claims when diagnosing RLS behavior.[5] [6]

Example diagnostic template:

```
begin;

set local role authenticated;

set local "request.jwt.claims" =
  '{"sub":"student-a-id","role":"authenticated"}';

-- Student A from tenant T1 tries to read a T2 record.
-- Expected: zero rows, with no sensitive data in error output.
select id, tenant_id, owner_id
from app.student_records
where id = 'record-owned-by-student-c-in-tenant-t2';

-- Expected: zero rows updated.
update app.student_records
set display_name = 'should-not-write'
where id = 'record-owned-by-student-c-in-tenant-t2';

rollback;
```

A zero-row result may be the correct RLS denial behavior. First prove the T2 fixture exists using an authorized T2 test account, and prove the T1 account can read its own authorized record; otherwise, an empty result might indicate bad fixtures or a globally broken policy rather than correct tenant isolation. The detailed generated walkthrough includes safe savepoint handling for expected rejected inserts, required evidence fields, side-effect checks, and remediation steps when a path fails.

⁂

1. https://cheatsheetseries.owasp.org/IndexASVS.html
2. https://cybersigmacs.com/knowledge-center/owasp-asvs/
3. https://docs.github.com/en/actions/reference/security/secure-use
4. https://docs.github.com/en/actions/reference/security/securely-using-pull_request_target
5. https://supabase.com/docs/guides/local-development/testing/overview
6. https://supabase.com/docs/guides/database/extensions/pgtap
7. https://www.guardlayer.io/blog/how-to-test-supabase-rls
8. https://ismailzahir.com/en/blog/github-actions-pull-request-target-security

9. https://gist.github.com/cekuu35/35ccb80e1fc986fab088c4b804211fbe
10. https://github.com/amannn/action-semantic-pull-request/issues/182
11. https://github.com/orgs/community/discussions/168221
12. https://quality.arc42.org/standards/owasp-asvs
13. https://www.appknox.com/cyber-security-jargons/owasp-asvs
14. https://github.com/adversis/owasp-asvs
15. https://www.linkedin.com/posts/freddymacho_owaspasvs500en-activity-7392538872309559298-kT6U
16. https://reversepolarity.be/blog/asvs-5-v1-v2-encoding-validation-and-why-the-order-matters/
17. https://sf2framework.com/07-relationships/owasp-asvs/
18. https://www.scribd.com/document/888136496/OWASP-Application-Security-Verification-Standard-5-0-0-En
19. https://adversis.github.io/owasp-asvs/
20. https://github.com/Adversis/owasp-asvs/blob/main/README.md
21. https://owasp.peteraim.com/en/asvs.html
22. https://supabase.com/docs/guides/database/postgres/row-level-security
23. https://makerkit.dev/blog/tutorials/supabase-rls-best-practices
24. https://dev.to/whoffagents/supabase-row-level-security-in-production-patterns-that-actually-work-2l78
25. https://owasp.org/projects/asvs
26. https://codific.com/owasp-asvs-a-comprehensive-overview/
27. https://www.aikido.dev/learn/compliance/compliance-frameworks/owasp-asvs
28. https://openssf.org/blog/2024/08/12/mitigating-attack-vectors-in-github-workflows/
29. https://www.stepsecurity.io/blog/defend-your-github-actions-ci-cd-environment-in-public-repositories
30. https://synacktiv.com/publications/github-actions-exploitation-untrusted-input